

Ressourcen

Ressourcen

- Ressourcen sind wiederverwendbare Elemente
 - Style
 - Referenz zu einem externen Objekt
 - Datenquelle
- Können an verschiedenen Orten hinterlegt werden:
 - aktuelles Element
 - aktuelles Dokument
 - Applikation
 - Bibliothek

Ressourcen sind Objekte, die in einer zentralen Sammlung gespeichert werden und von verschiedenen Teilen der Anwendung referenziert werden können. Sie dienen dazu, wiederverwendbare Elemente wie Styles, Templates, Bilder oder andere Objekte zu definieren und zentral zu verwalten.

Ressourcen werden in XAML definiert und können, je nach Position ihrer Definition, von allen Bereichen der Applikation verwendet werden.

Vorteile von Ressourcen

- Effizienteres Arbeiten am Code
 - Objekt einmal definieren und öfters weiterverwenden
- Wartbarkeit wird erhöht
 - Änderungen an einer Ressource (z.B. einem Style) können an einem Ort durchgeführt werden
- Dynamik
 - Ressourcen können separat von der Applikation gespeichert werden
 - Skins

Die Verwendung von Ressourcen bietet mehrere Vorteile. Sie ermöglichen eine konsistente Gestaltung und Darstellung der Benutzeroberfläche, da Styles und Templates zentral definiert und auf verschiedene Controls angewendet werden können. Ressourcen fördern auch die Wiederverwendbarkeit von Code und erleichtern die Wartung und Anpassung der Anwendung, da Änderungen an einer Ressource automatisch auf alle Stellen übertragen werden, an denen sie verwendet wird. Auch sorgen sie für eine effizientere Verwendung des Arbeitsspeichers, da Objekte wie Converter und Validierungsklassen bei mehrfacher Verwendung nicht mehrfach instanziiert werden müssen.

Ressourcen anlegen

- App, Window, Layoutcontainer und Steuerelemente haben Ressourcen-Eigenschaften vom Datentyp ResourceDictionary
- Stellen jeweils Ressourcen für den entsprechenden Bereich zur Verfügung

```
<Window.Resources>
  <sys:Double x:Key="globalFontSize">25.0</sys:Double>
  <SolidColorBrush Color="LightBlue" x:Key="Scb_DefaultBrush"/>
</Window.Resources>
```

- Ressourcen benötigen x:Key zur Identifizierung
 - Innerhalb der selben Resources-Eigenschaft muss der Key einzigartig sein
 - Bei Namenskonflikten (über mehrere Ebenen hinweg) wird immer die lokalste Ressource verwendet

Ressourcen können in jedem Bereich von XAML etabliert werden (App, Window, Layoutcontainer, Controls, ...) und werden jeweils für diesen Bereich zur Verfügung gestellt. Dafür werden die ‚Resources‘-Eigenschaften der Objekte verwendet. Hierin befindet sich ein Objekt vom Typ ‚ResourceDictionary‘, welches in XAML befüllt werden kann. Als Ressource kann jedes beliebige instanziierte Objekt verwendet werden.

Jeder Ressource muss (bis auf wenige Ausnahmen) über die Eigenschaft ‚x:Key‘ ein eindeutiger Indentifikator zugeordnet werden, der in dem resourceDictionary nur einmal vorkommen kann. Da aber über verschiedene ResourceDictionaries hinweg durchaus auch gleiche Keys vorkommen können, wird bei Namenskonflikten immer die Ressource verwendet, welche ‚am nächsten am Verwender drann ist‘, d.h. auf welche bei rückwärtigem Durchlaufen des Logical Trees als erstes getroffen wird.

Ressourcenzugriff

- Der Zugriff erfolgt über Ressourcen-Bindungen in der MarkupExtension-Syntax
 - **StaticResource**: statische Bindung -> Laufzeit-Veränderungen der Ressource haben keine Auswirkungen
`<Button Background="{StaticResource Scb_Default}" />`
 - **DynamicResource**: dynamische Bindung -> Eigenschaft übernimmt Laufzeit-Veränderung der Ressource, benötigt aber vergleichsweise viel Rechenleistung
`<Button Background="{DynamicResource Scb_Default}" />`
 - **x:Static**: statische Bindung an Nicht-Xaml-Ressourcen, wie z.B. resx-Dateien, statische Eigenschaften oder Enum-Werte
`<Button Content="{x:Static local:Strings.Btn_Content}" />`

Der Zugriff auf Ressourcen erfolgt über spezielle Bindungs-Objekte in der MarkupExtension-Syntax. Hierzu werden diese Objekte unter Angabe des Ressourcen-Keys in die Eigenschaften eingelegt. Mögliche Bindungsobjekte sind:

1. **StaticResource**: Eine statische Bindung, welche bei Objektinitialisierung die Ressource lädt und danach nicht mehr überwacht. Laufzeitveränderungen der Ressource werden nicht beachtet.
2. **DynamicResource**: Eine dynamische Bindung, welche kontinuierlich die Ressource auf Veränderungen überprüft. Damit werden zwar Laufzeitveränderungen übernommen, allerdings benötigt diese Bindung dafür vergleichsweise viel mehr Rechenleistung. Eine übermäßige Verwendung dieser Bindung kann sich in Performanceverlust äußern.
3. **x:Static**: Eine spezielle statische Bindung zur Anbindung an Nicht-XAML-Ressourcen. Sie wird insbesondere bei der Lokalisierung (siehe entsprechender Abschnitt) zur Anbindung an resx-Dateien verwendet. Hierüber können aber auch statische Eigenschaften von Objekt-Klassen sowie spezifische Enum-Werte verknüpft werden.

ResourceDictionaries

- Sammlungen von Ressourcen

- Ressourcen werden mit Key - Value paar abgelegt (.NET Dictionary)
- Auch Verwendung mehrerer Dictionaries durch Zusammenfügen

```
<ResourceDictionary.MergedDictionaries>
```

- Können auch als separate Datei angelegt werden

- Datei muss auf der Ebene angemeldet werden, für welche die Ressourcen bereitgestellt werden sollen

```
<ResourceDictionary Source="Dictionary1.xml"/>
```

Eine Ressourcen-Eigenschaft kann ein ResourceDictionary-Objekt beinhalten. Hierin werden die Ressourcen als Key-Value-Paar abgelegt. In C# werden diese wie Objekte vom Typ Dictionary<TKey, TValue> verwendet. Über die Eigenschaft ‚MergedDictionaries‘ können mehrere ResourceDictionary-Objekte zusammengefasst werden.

ResourceDictionaries können auch als separate Dateien angelegt werden. Diese haben dann das Root-Element ‚ResourceDictionary‘ und keine Code-Behind-Datei. Diese separaten ResourceDictionaries müssen dann in der Resource-Eigenschaft nach dem Schema `<ResourceDictionary Source=„relativerDateipfad“/>` angemeldet werden, für deren Bereich die Ressourcen zur Verfügung gestellt werden sollen.

Styles

Styles

- Ein Style ist eine Sammlung von Property-Values für einen spezifischen Control-Typ
 - TargetType gibt Control-Typ an
 - Werte werden als neue ‚Default‘-Werte für die Eigenschaften gesetzt
 - Praktisch für immer wiederkehrende gleiche Eigenschaften eines Controls
 - Bsp.: Margin, Background, Allignment, Padding, Font,
- Controls implementieren Styles über die ‚Style‘-Eigenschaft

```
<Button>
  <Button.Style>
    <Style TargetType="Button">
      <Setter Property="Background" Value="Yellow" />
      <Setter Property="FontSize" Value="{StaticResource globalFontSize}" />
    </Style>
  </Button.Style>
</Button>
```

Styles sind eine Möglichkeit, das Erscheinungsbild von Controls und Elementen in der Benutzeroberfläche einheitlich zu gestalten. Ein Style ist eine Sammlung von Eigenschaftswerten, die auf ein oder mehrere Controls angewendet werden können, wodurch diese Werte zu den neuen ‚Standartwerten‘ der Eigenschaften werden. Die ‚TargetType‘-Eigenschaft definiert den Control-Typ, für den der Style zuständig ist.

Ein Style definiert dabei verschiedene Eigenschaften wie z.B. Hintergrundfarbe, Schriftart, Schriftgröße, Ausrichtung und weitere visuelle Aspekte eines Controls. Durch die Anwendung eines Styles auf ein Control können diese Eigenschaften automatisch gesetzt werden, ohne dass sie explizit in jedem einzelnen Control definiert werden müssen.

Styles werden in XAML definiert und können auf verschiedene Arten angewendet werden. Sie können entweder direkt in einem Control über die Eigenschaft "Style" zugewiesen werden, oder sie können in den Ressourcen der Anwendung definiert und dann über die Ressourcensyntax referenziert werden.

Die Verwendung von Styles bietet mehrere Vorteile. Sie ermöglichen eine einheitliche Gestaltung der Benutzeroberfläche, da das Erscheinungsbild von Controls zentral definiert und auf verschiedene Stellen in der Anwendung angewendet werden kann. Styles fördern die Wiederverwendbarkeit von Gestaltungsregeln und erleichtern die Wartung und Anpassung der Anwendung, da Änderungen an einem Style automatisch auf alle Stellen übertragen werden, an denen er verwendet wird.

Styles als Ressourcen

- Styles sind die häufigsten Ressourcen-Arten
- Lokal (mit Key): Controls müssen den Style über Ressourcen-Bindung explizit implementieren
- Global (ohne Key): Controls implementieren automatisch den nächsten Style ihres Datentypen
- Styles können über ‚BasedOn‘ von anderen Styles erben:

```
<Style TargetType="Button" x:Key="Sty_Btn_02" BasedOn="{StaticResource Sty_Btn_01}">
```
- Styles können auch von globalen Styles erben, wenn der Datentyp angegeben wird

```
<Style TargetType="Button" x:Key="Sty_Btn_03" BasedOn="{StaticResource {x:Type Button}}">
```

Werden Styles als Ressourcen gesetzt, was die häufigste Verwendung von Ressourcen darstellt, kann diesen, wie bei anderen Ressourcen, ein x:Key zugeordnet werden, über den eine explizite statische oder dynamische Ressourcenbindung durchgeführt werden kann.

Alternativ kann bei Styles ausnahmsweise auf einen x:Key verzichtet werden. Es wird dann automatisch der verwendete TargetType als Key gesetzt. Diese sogenannten globalen Styles werden dann automatisch allen Controls dieses Datentypen in dem entsprechenden Bereich zugeordnet.

Ein Style kann auch von einem anderen Style erben. Hierfür wird die ‚BasedOn‘-Eigenschaft in Kombination mit einer Ressourcenbindung verwendet. Dies erfolgt unter Angabe des x:Keys des Mutterstyles oder, bei globalen Styles, unter Angabe der x:Type-Instanz des Control-Datentypen.

Manipulation von Styles

- Styles werden normalerweise im Markup definiert
- Können zur Laufzeit ausgetauscht werden -> DynamicResource

```
MainStack.Resources["ButtonStyle"] = new Style(){...};
```

Wie auch andere Ressourcen, können auch Styles zur Laufzeit manipuliert werden. Wenn dies innerhalb eines Ressourcenfelds passiert müssen die Controls über eine DynamicResource-Bindung angebunden sein, damit diese auf die Veränderung reagieren.

Themes

- Sammlungen von Styles
- Werden global deklariert und automatisch von den UI-Elementen übernommen

```
<Application.Resources>
  <ResourceDictionary>
    <ResourceDictionary.MergedDictionaries>
      <ResourceDictionary Source="Dictionary1.xaml"/>
      <ResourceDictionary Source="/PresentationFramework.Luna;component/themes/Luna.Homestead.xaml"/>
    </ResourceDictionary.MergedDictionaries>
  </ResourceDictionary>
</Application.Resources>
```



Themes sind eine Möglichkeit, das Erscheinungsbild einer WPF-Anwendung global zu ändern, indem vordefinierte Stilvorlagen und Ressourcen verwendet werden. Ein Theme definiert eine Sammlung von Styles, Templates und Ressourcen, die das Aussehen und Verhalten von Controls und Elementen in der Benutzeroberfläche beeinflussen.

Themes können auf Anwendungsebene festgelegt werden und beeinflussen somit alle Controls und Elemente in der gesamten Anwendung. Ein Theme kann verschiedene visuelle Aspekte wie Farben, Schriftarten, Schriftgrößen, Hintergrundbilder und weitere Eigenschaften definieren, die das Erscheinungsbild der Anwendung beeinflussen. Durch die Anwendung eines Themes können Entwickler das Erscheinungsbild ihrer Anwendung schnell und einfach anpassen, ohne dabei jeden einzelnen Style oder jedes Control manuell bearbeiten zu müssen.

WPF bietet verschiedene vordefinierte Themes wie z.B. das Standard-WPF-Theme, das Aero-Theme, das Luna-Theme und das Metro-Theme. Diese Themes können verwendet werden, um das Aussehen und Verhalten der Anwendung entsprechend den Designrichtlinien von Windows anzupassen. Darüber hinaus besteht auch die Möglichkeit, eigene Themes zu erstellen und zu verwenden, indem man individuelle Stilvorlagen und Ressourcen definiert und sie als Theme-Ressourcen in der Anwendung registriert.